Motivation and Background

- Databases evolve over time as a result of *transactions*, whose purpose is to update the database with new information.
- Example: An educational database might have a transaction specifically designed to change a student's grade.
  - This would normally be a procedure which, when invoked on a specific student and grade, first checks that the database satisfies certain preconditions (e.g., that there is a record for the student, and that the new grade differs from the old), and if so, records the new grade.
- This is a *procedural* notion. Transactions also *physically modify* the database.
- Ideally, we want a *specification* of the entire evolution of a database.
- This section proposes such a specification of database transactions by appealing to various ideas from the AI planning literature:
  - The situation calculus.
  - The frame problem.
  - The projection problem in planning.
  - Goal regression for plan synthesis.
- Theory of database updates $\cong$ AI planning in the situation calculus.

Database Updates: A Proposal

- Represent databases in the situation calculus. Updatable relations are fluents, i.e. they take a situation argument.
- Treat update transactions exactly like actions in the AI planning domain. Transactions are functions.
- For this to work, need a solution to the frame problem.

The Basic Approach: An Example

An Education Database

- **Relations**
  1. $enrolled(st, course, s)$: st is enrolled in $course$ when the database is in situation s.
  2. $grade(st, course, grade, s)$: The grade of st in $course$ is $grade$ when the database is in situation s.
  3. $prerequ(pre, course)$: pre is a prerequisite course for $course$.
- **Initial Database State**
  These will be arbitrary first order sentences, the only restriction being that fluents mention only the initial situation S_0.

$$enrolled(Sue, C100, S_0) \vee enrolled(Sue, C200, S_0),$$

$$(\exists c) enrolled(Bill, c, S_0),$$

$$(\forall p).prerequ(p, P300) \equiv p = P100 \vee p = M100,$$

$$(\forall p) \neg prerequ(p, C100),$$

$$(\forall c).enrolled(Bill, c, S_0) \equiv c = M100 \vee c = C100 \vee c = P200,$$

$$enrolled(Mary, C100, S_0), \quad \neg enrolled(John, M200, S_0), \ldots$$

$$grade(Sue, P300, 75, S_0), \quad grade(Bill, M200, 70, S_0), \ldots$$

Example: Continued

- **Database Transactions**
  - Denote by function symbols.
  - Treat exactly like actions in situation calculus planning.
- For the example, there are three transactions:
  1. $register(st, c)$,
  2. $change(st, c, g)$,
  3. $drop(st, c)$.
- A student can register in a course iff she has obtained a grade of at least 50 in all prerequisites for the course:

$$Poss(register(st, c), s) \equiv \{(\forall p).prerequ(p, c) \supset (\exists g).grade(st, p, g, s) \wedge g \geq 50\}.$$

- It is possible to change a student's grade iff he has a grade which is different than the new grade:

$$Poss(change(st, c, g), s) \equiv (\exists g').grade(st, c, g', s) \wedge g' \neq g.$$

- A student may drop a course iff the student is currently enrolled in that course:

$$Poss(drop(st, c), s) \equiv enrolled(st, c, s).$$

Example: Continued

- **Transaction Effect Axioms:**

  $\neg enrolled(st, c, do(drop(st, c), s)),$

  $enrolled(st, c, do(register(st, c), s)),$

  $grade(st, c, g, do(change(st, c, g), s)),$

  $g' \neq g \supset \neg grade(st, c, g', do(change(st, c, g), s)).$

  Solve the frame problem.

- **Successor state axioms:**

  $$enrolled(st, c, do(a, s)) \equiv a = register(st, c) \ \vee \\ enrolled(st, c, s) \wedge a \neq drop(st, c),$$

  $$grade(st, c, g, do(a, s)) \equiv a = change(st, c, g) \ \vee \\ grade(st, c, g, s) \wedge (\forall g')a \neq change(st, c, g').$$

Queries

- All updates are $virtual$; the database is never physically changed.
- Querying the database resulting from a sequence of update transactions:
  "Is John enrolled in any courses after the transaction sequence $drop(John, C100), register(Mary, C100)$ has been 'executed'?"

  $$Database \models (\exists c).enrolled(John, c, do(register(Mary, C100), do(drop(John, C100), S_0))).$$

- This is the $projection\ problem$ in AI planning. More later.
- Such a sequence of update transactions is called a database log in database theory.
- Query evaluation wrt a database log: later.

14.

Actions

Situation calculus

The situation calculus is a dialect of FOL for representing dynamically changing worlds in which all changes are the result of named actions.

There are two distinguished sorts of terms:

- actions, such as
  - put(x,y) put object x on top of object y
  - walk(loc) walk to location loc
  - pickup(r,x) robot r picks up object x
- situations, denoting possible world histories. A distinguished constant S_0 and function symbol do are used
  - S_0 the initial situation, before any actions have been performed
  - $do(a,s)$ the situation that results from doing action a in situation s

for example: $do(\text{put}(A,B),do(\text{put}(B,C),S_0))$ the situation that results from putting A on B after putting B on C in the initial situation

Fluents

Predicates or functions whose values may vary from situation to situation are called fluents.

These are written using predicate or function symbols whose last argument is a situation

for example: $\text{Holding}(r, x, s)$: robot r is holding object x in situation s

can have: $\neg\text{Holding}(r, x, s) \wedge \text{Holding}(r, x, do(\text{pickup}(r,x),s))$

the robot is not holding the object x in situation s, but is holding it in the situation that results from picking it up

Note: there is no distinguished "current" situation. A sentence can talk about many different situations, past, present, or future.

A distinguished predicate symbol $Poss(a,s)$ is used to state that a may be performed in situation s

for example: $Poss(\text{pickup}(r,x), S_0)$

it is possible for the robot r to pickup object x in the initial situation

This is the entire language.

Preconditions and effects

It is necessary to include in a KB not only facts about the initial situation, but also about world dynamics: what the actions do.

Actions typically have preconditions: what needs to be true for the action to be performed

- $Poss(\text{pickup}(r,x), s) \equiv \forall z.\ \neg\text{Holding}(r,z,s) \wedge \neg\text{Heavy}(x) \wedge \text{NextTo}(r,x,s)$

  a robot can pickup an object iff it is not holding anything, the object is not too heavy, and the robot is next to the object

  Note: free variables assumed to be universally quantified

- $Poss(\text{repair}(r,x), s) \equiv \text{HasGlue}(r,s) \wedge \text{Broken}(x,s)$

  it is possible to repair an object iff the object is broken and the robot has glue

Actions typically have effects: the fluents that change as the result of performing the action

- $\text{Fragile}(x) \supset \text{Broken}(x, do(\text{drop}(r,x),s))$

  dropping a fragile object causes it to break

- $\neg\text{Broken}(x, do(\text{repair}(r,x),s))$

  repairing an object causes it to be unbroken

The frame problem

To really know how the world works, it is also necessary to know what fluents are *unaffected* by performing an action.

- $\text{Colour}(x,c,s) \supset \text{Colour}(x, c, do(\text{drop}(r,x),s))$

  dropping an object does not change its colour

- $\neg\text{Broken}(x,s) \wedge [\, x \neq y \vee \neg\text{Fragile}(x)\,] \supset \neg\text{Broken}(x, do(\text{drop}(r,y),s)$

  not breaking things

These are sometimes called frame axioms.

Problem: need to know a vast number of such axioms. (Few actions affect the value of a given fluent; most leave it invariant.)

an object's colour is unaffected by picking things up, opening a door, using the phone, turning on a light, electing a new Prime Minister of Canada, *etc.*

The frame problem:

- in building KB, need to think of these $\sim 2 \times A \times F$ facts about what does not change
- the system needs to reason efficiently with them

What counts as a solution?

- Suppose the person responsible for building a KB has written down *all* the effect axioms

  for each fluent F and action A that can cause the truth value of F to change, an axiom of the form $[R(s) \supset \pm F(do(A,s))]$, where $R(s)$ is some condition on s

- We want a systematic procedure for generating all the frame axioms from these effect axioms
- If possible, we also want a *parsimonious* representation for them (since in their simplest form, there are too many)

Why do we want such a solution?

- frame axioms are necessary to reason about actions and are not entailed by the other axioms
- convenience for the KB builder
  - modularity: only add effect axioms
  - accuracy: no inadvertent omissions
- for theorizing about actions

The projection task

What can we do with the situation calculus?

We will see later that it can be used for planning.

A simpler job we can handle directly is called the projection task.

> Given a sequence of actions, determine what would be true in the situation that results from performing that sequence.

This can be formalized as follows:

> Suppose that $R(s)$ is a formula with a free situation variable s.
>
> To find out if $R(s)$ would be true after performing $\langle a_1,...,a_n \rangle$ in the initial situation, we determine whether or not
>
> $$KB \models R(do(a_n,do(a_{n-1},...,do(a_1,S_0)...)))$$

For example, using the effect and frame axioms from before, it follows that $\neg \text{Broken}(\text{B},s)$ would hold after doing the sequence

$$\langle \text{pickup(A), pickup(B), drop(B), repair(B), drop(A)} \rangle$$

The legality task

The projection task above asks if a condition would hold after performing a sequence of actions, but not whether that sequence can in fact be properly executed.

We call a situation legal if it is the initial situation or the result of performing an action whose preconditions are satisfied starting in a legal situation.

The legality task is the task of determining whether a sequence of actions leads to a legal situation.

This can be formalized as follows:

> To find out if the sequence $\langle a_1,...,a_n \rangle$ can be legally performed in the initial situation, we determine whether or not
>
> $$KB \models Poss(a_i, do(a_{i-1},...,do(a_1,S_0)...))$$
>
> for every i such that $1 \leq i \leq n$.

Limitations of the situation calculus

This version of the situation calculus has a number of limitations:

- no time: cannot talk about how long actions take, or when they occur
- only known actions: no hidden exogenous actions, no unnamed events
- no concurrency: cannot talk about doing two actions at once
- only discrete situations: no continuous actions, like pushing an object from A to B.
- only hypotheticals: cannot say that an action has occurred or will occur
- only primitive actions: no actions made up of other parts, like conditionals or iterations

We will deal with the last of these below.

First we consider a simple solution to the frame problem ...